

Frozen CLIP plus a small side network

Taking the DFD-FCG deepfake detector (CVPR 2025) to CIFAR-10: what we built, what we measured, and what it means

OUTLINE

Four parts in this talk

1

The paper

What DFD-FCG does,
and why it keeps CLIP
frozen

2

Our CIFAR-10 version

What we kept, what we
changed, and why

3

Results

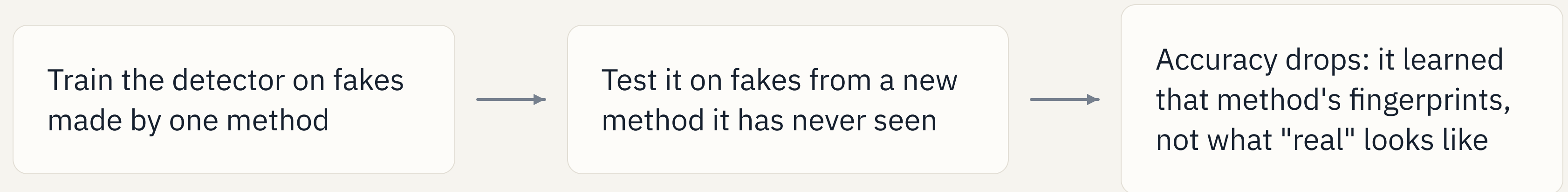
96.42% on 10,000 test
images, and how we
checked it

4

Extra checks

Baselines, and a real-vs-
fake test on CIFAKE

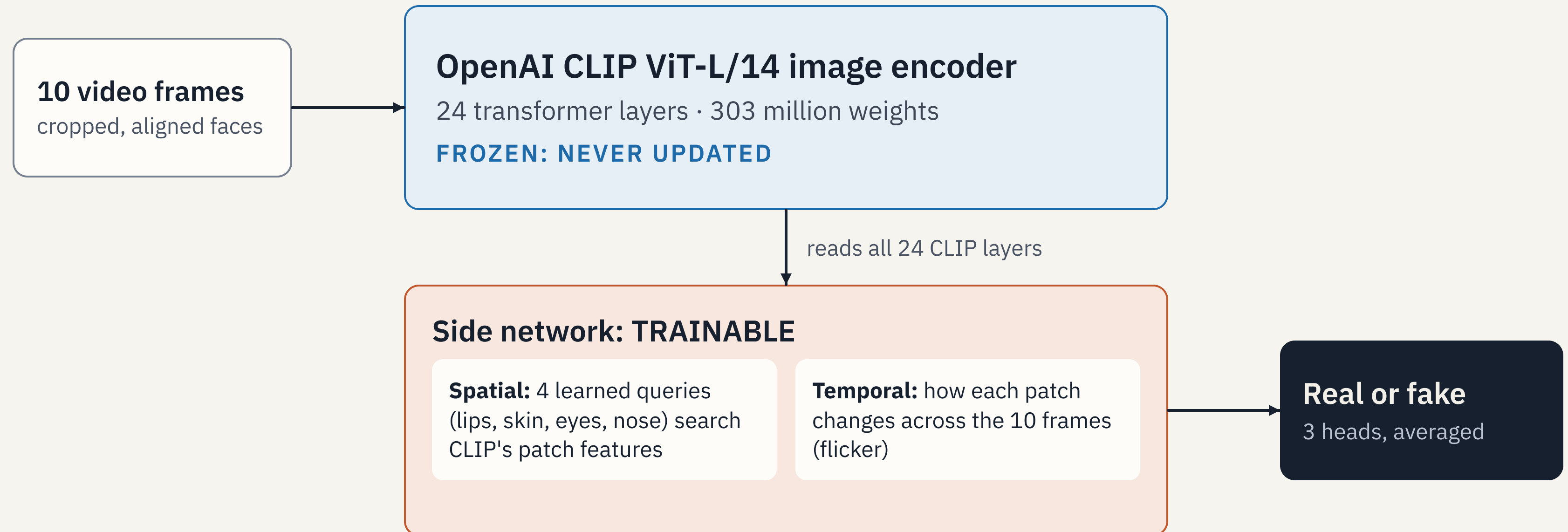
Deepfake detectors break on unseen fakes



The paper's answer

Start from OpenAI's CLIP, trained on 400 million image–text pairs, so it already knows what normal photos and faces look like. Keep it frozen, and train only a small add-on network that looks at the face regions where fakes usually break.

The paper's idea: frozen CLIP + side network



The parts, as the official code builds them

Backbone

CLIP ViT-L/14 from OpenAI. Frozen: it only turns each frame into features.

Side network

One small block next to each of the 24 CLIP layers. The only part that learns.

Spatial branch

4 query vectors, one per face part, pick out matching patches from CLIP's features.

Face guidance (FCG)

Queries start from real lips/skin/eyes/nose features; an extra loss keeps each on its part.

Temporal branch

Compares every patch across the 10 frames to catch flicker between frames.

Output & training

3 heads averaged → real or fake. Focal loss, 30 epochs, learning rate 0.0001.

The authors' model, running on my laptop



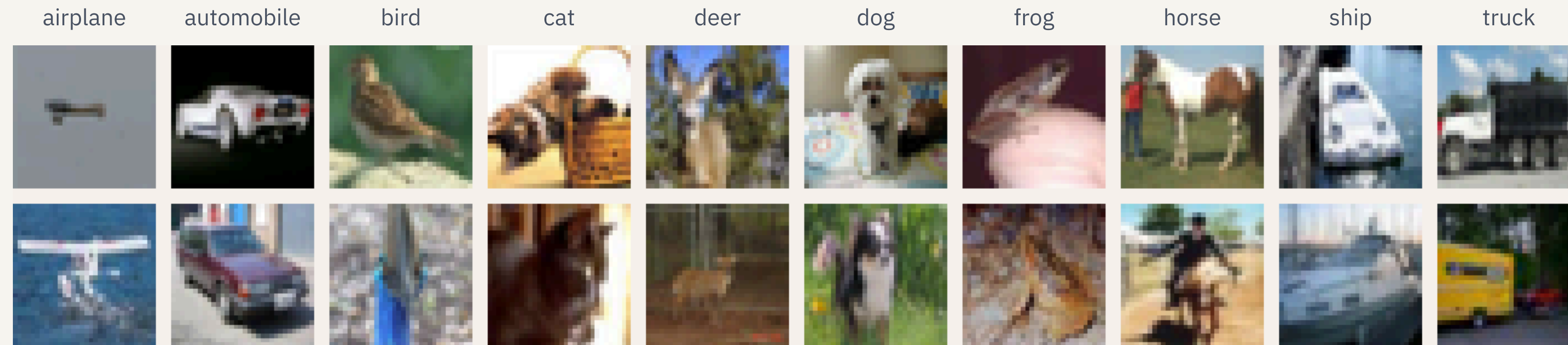
Real video → predicted REAL (fake score 0.45)



Face-swapped video → predicted FAKE (fake score 0.93)

2 · OUR CIFAR-10 VERSION

The task: the same idea on CIFAR-10



60,000
colour images

32×32
pixels each: very small

10
classes, 6,000 images each

50k / 10k
train / test split

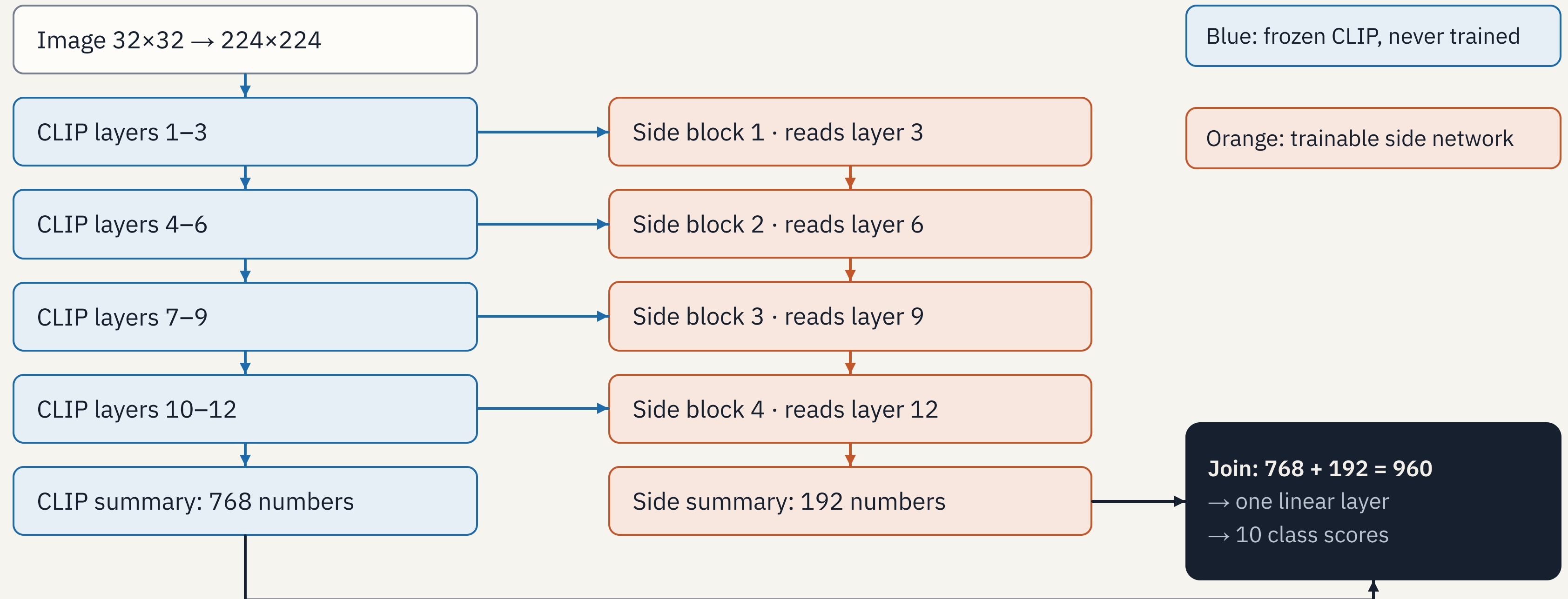
What we kept, what we changed

Part	DFD-FCG paper	Our CIFAR-10 version	Why
Input	10 face frames	1 image, 32 → 224 px	no video in CIFAR-10
Backbone	CLIP ViT-L/14, frozen	CLIP ViT-B/16, frozen	4 GB laptop GPU
Side network	a block at all 24 layers	4 blocks (3, 6, 9, 12)	lighter, same idea
Face guidance	lips, skin, eyes, nose	removed	no faces
Frame branch	compares 10 frames	removed	single images
Output	real or fake	10 classes	CIFAR-10 labels
Loss	focal + guidance loss	cross-entropy	standard for 10 classes

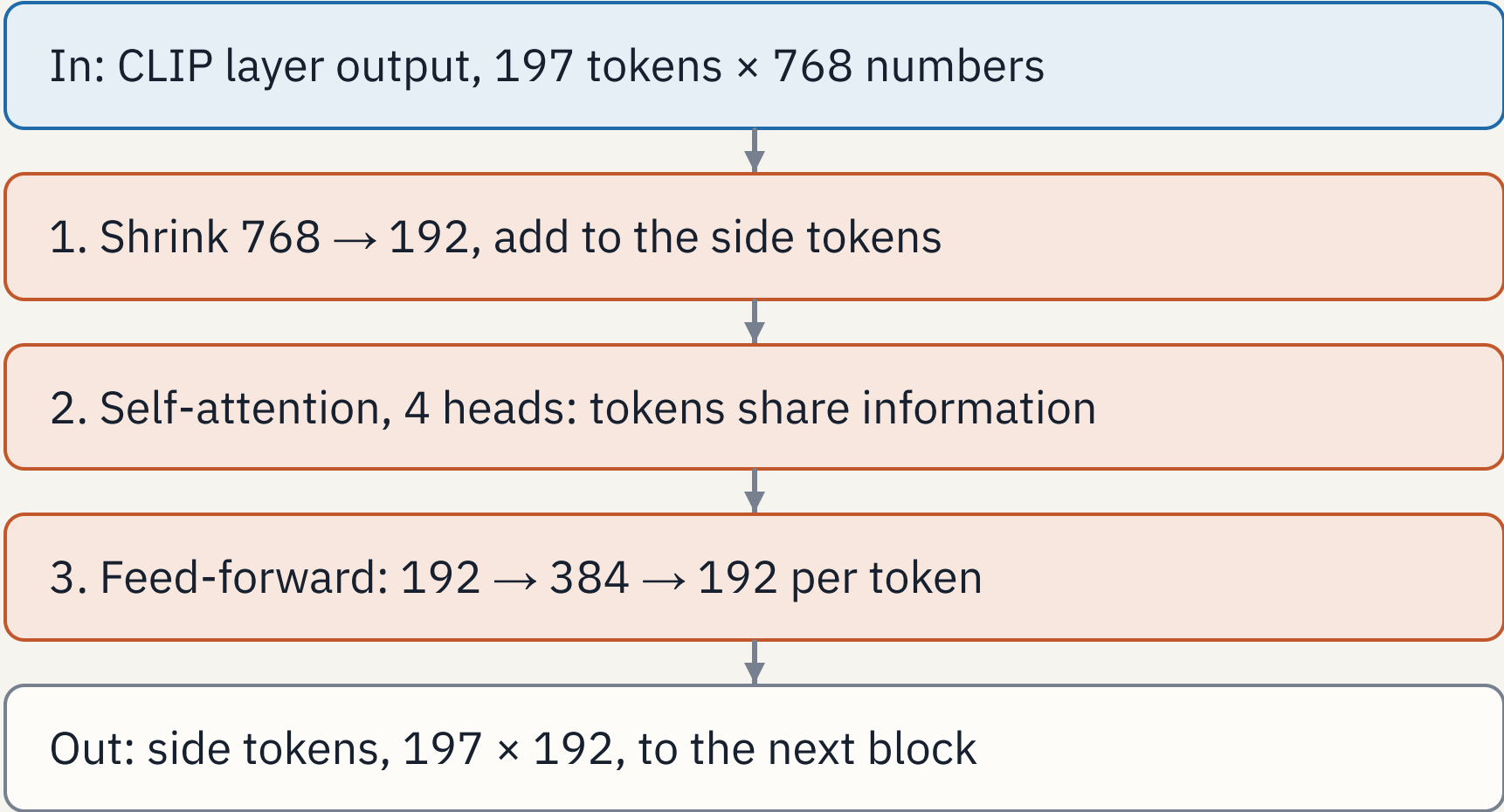
Kept: CLIP stays frozen, and a small trainable side network reads CLIP's middle layers. Only about 2% of the weights learn.

2 · OUR CIFAR-10 VERSION

Our model, block by block



Inside one side block: a tiny transformer



Steps 2 and 3 each start with a LayerNorm and add their result back to the input (residual connection).

Part of the block	Weights
Shrink 768 → 192	147,648
Self-attention	148,224
Feed-forward	148,032
2 LayerNorms	768
One block	444,672
× 4 blocks	1,778,688
Final layer 960 → 10	9,610
All trainable	1,788,298

2 · OUR CIFAR-10 VERSION

Only 2% of the weights learn

86.2M

frozen CLIP weights (86,193,152)

1.79M

trained weights (1,788,298)

2.03%

of all weights are trained

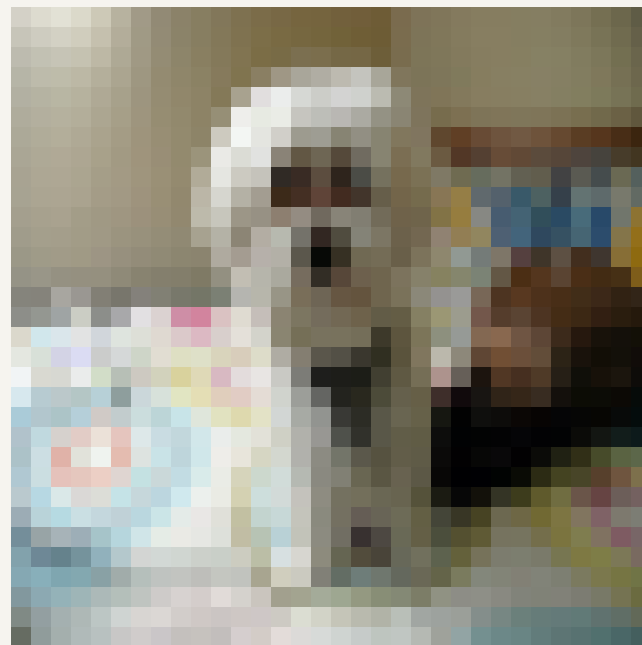


The bar is to scale: blue = frozen CLIP, the thin orange end = everything we train.

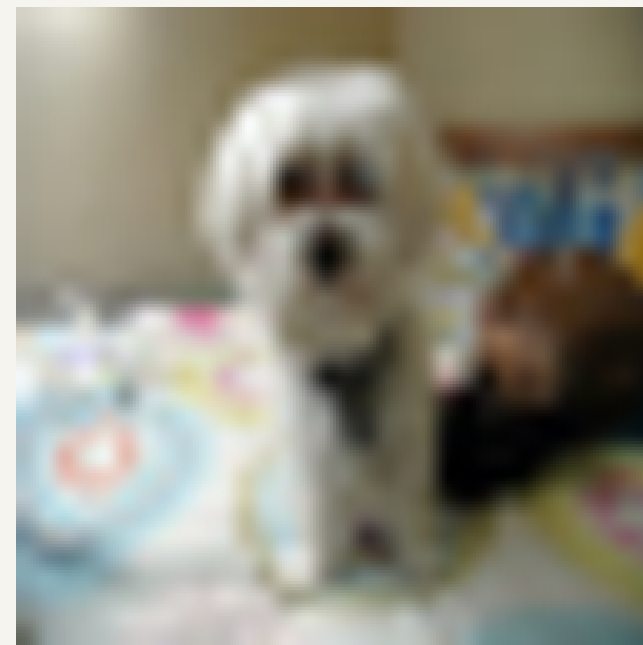
Saved model file: 352 MB, because it also stores a copy of frozen CLIP. The trainable part alone is about 7 MB.

2 · OUR CIFAR-10 VERSION

From a 32-pixel photo to 197 CLIP tokens



32 × 32 (as stored)



224 × 224 (bicubic)

1. Resize 32 → 224 with bicubic interpolation, as CLIP's own preprocessing does
2. Normalise the colours with CLIP's mean and standard deviation
3. Cut into 16×16 patches: $224 / 16 = 14$, so $14 \times 14 = 196$ patches
4. Add one summary token (CLS): 197 tokens
5. Inside CLIP ViT-B/16 every token is 768 numbers

How we trained it

Training data	50,000 images (5,000 per class)
Test data	10,000 images, never used for training
Batch size	128 images → 391 steps per epoch
Epochs	5 (one epoch = every training image seen once)
Optimizer	AdamW, learning rate 0.001, weight decay 0.0001
Schedule	cosine: learning rate falls smoothly from 0.001 towards 0
Loss	cross-entropy over the 10 classes
Augmentation	random horizontal flip (half the images mirrored)
Precision	mixed 16-bit (AMP) to save memory and time
Hardware, time	RTX 3050 Ti laptop GPU, 4 GB · 84 minutes in total

3 · RESULTS

First try 67.48%, after one fix 96.42%

RUN 1 · 3 EPOCHS

67.48%

- Classifier saw only the side network: 192 numbers, averaged over all tokens
- CLIP's own summary was thrown away

```
feat = side_tokens.mean(dim=1)
classifier = nn.Linear(192, 10)
```

RUN 2 · 5 EPOCHS

96.42%

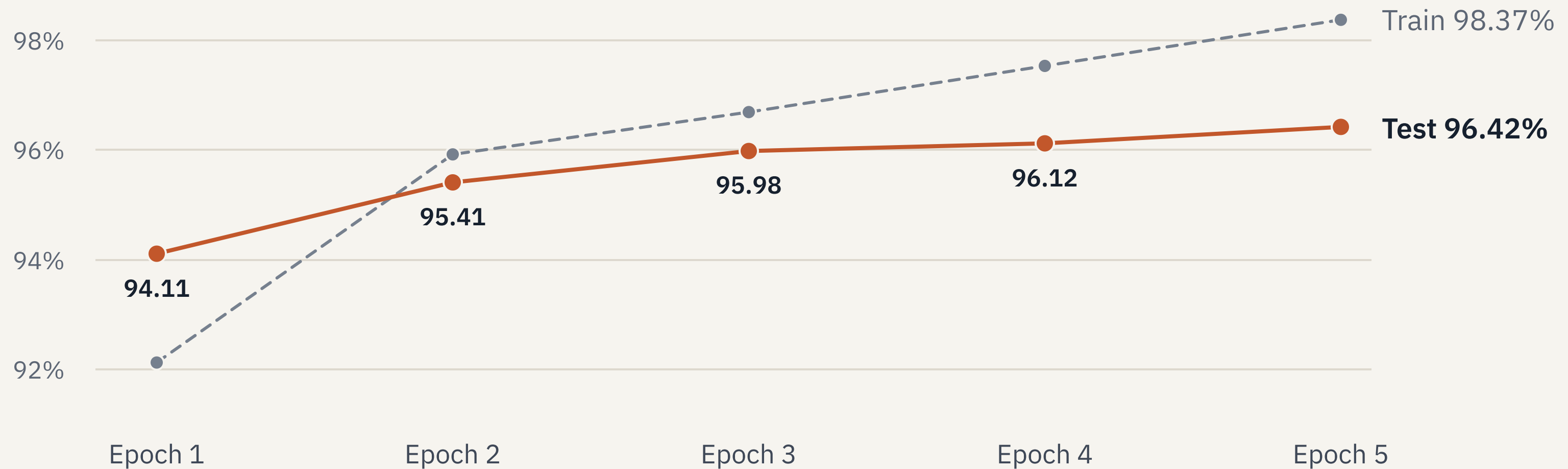
- Classifier sees CLIP's summary (768) and the side network's CLS token (192)
- Also: bicubic resize, batch 128

```
feat = torch.cat([clip_cls, side_cls], -1)
classifier = nn.Linear(768 + 192, 10)
```

The side network, starting from random numbers, could not replace what CLIP already knew. Letting the classifier see CLIP's own summary fixed it.

3 · RESULTS

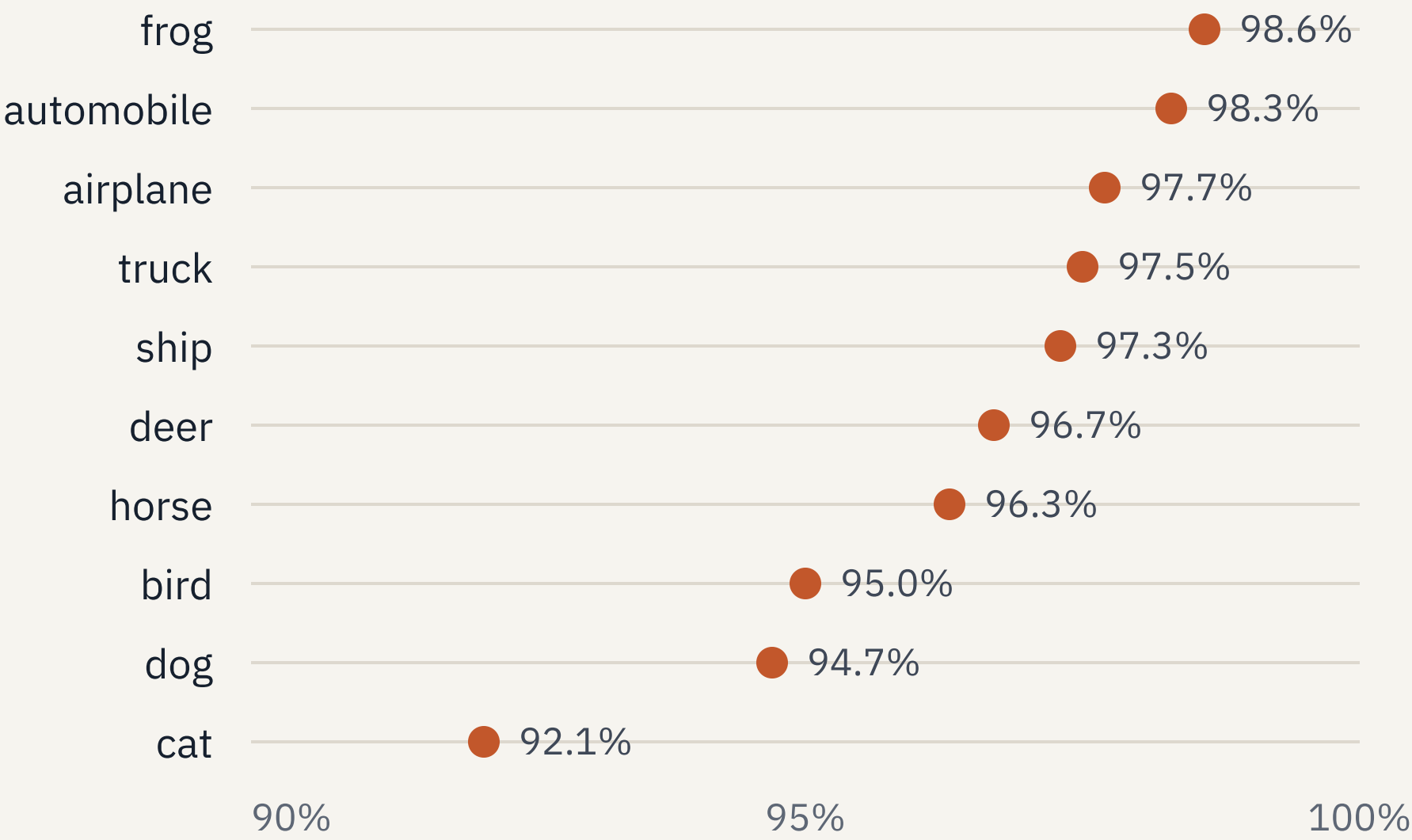
Test accuracy after each epoch



94.11% after the first epoch. At the end, train is 1.95 points above test: slight overfitting, while test accuracy still creeps up.

3 · RESULTS

Per class: cats and dogs are the hard pair



Most common mistakes

True → predicted	Count
cat → dog	56
dog → cat	39
ship → airplane	23
truck → automobile	19
horse → deer	18

Cat ↔ dog alone is 95 of the 358 mistakes (27%).

Is 96.42% real? Three checks

9,642

Re-tested the saved model

Loaded the saved weights again and ran all 10,000 test images: 9,642 correct = 96.42%. Every per-class number matches the training log.

0

Test images used in training

Official 50,000 / 10,000 split. Test images only go through the model without gradients, and no setting was tuned on them.

95.35%

Frozen CLIP alone gets close

A single linear layer on CLIP's frozen summary already reaches 95.35%, so 96.42% is in the expected range.

How much comes from CLIP itself?

Zero-shot CLIP: no training at all

0 weights trained



Linear probe: frozen CLIP + one linear layer

7,690 weights trained



Ours: frozen CLIP + side network

1,788,298 weights trained



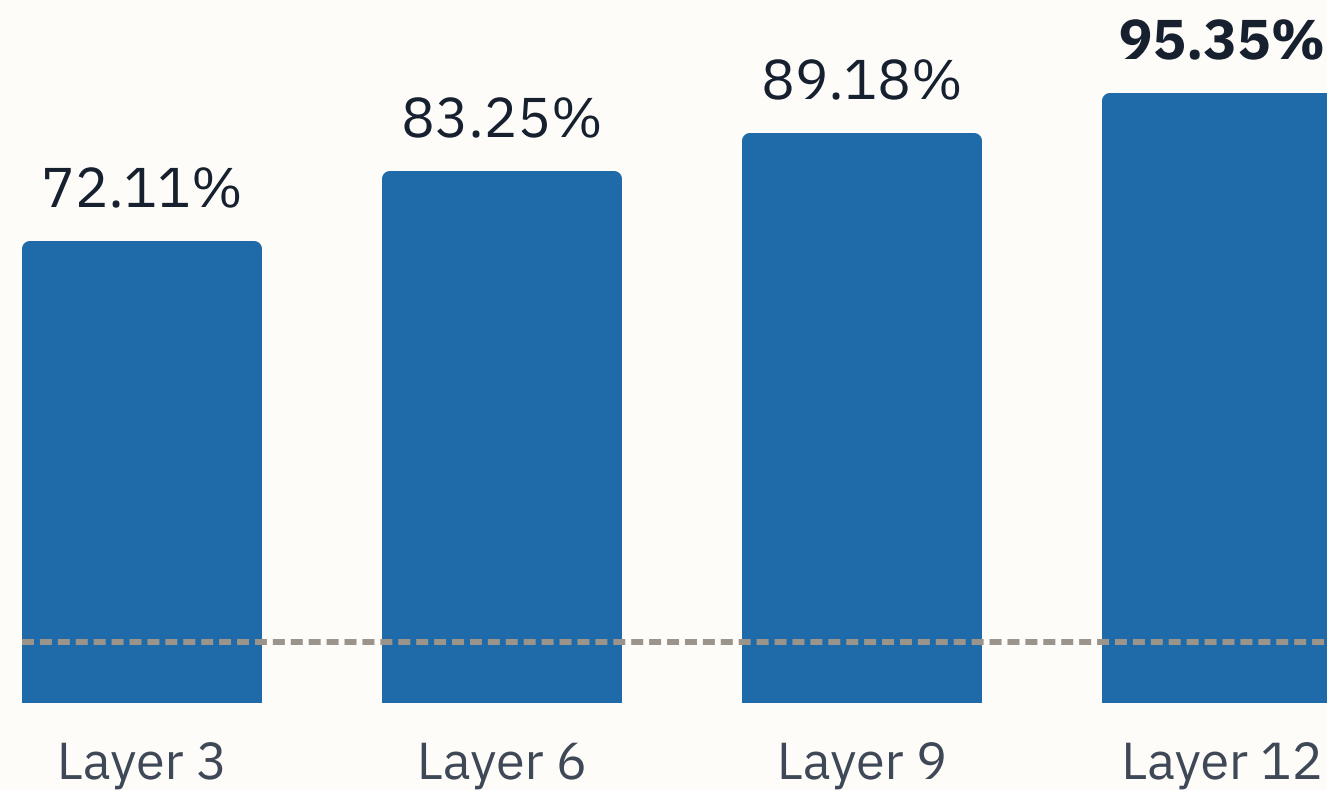
On CIFAR-10 most of the accuracy comes from CLIP's frozen features. The side network adds **+1.07 points** over the linear probe, 23% fewer errors (465 → 358).

4 · EXTRA CHECKS

Which CLIP layers hold the answer?

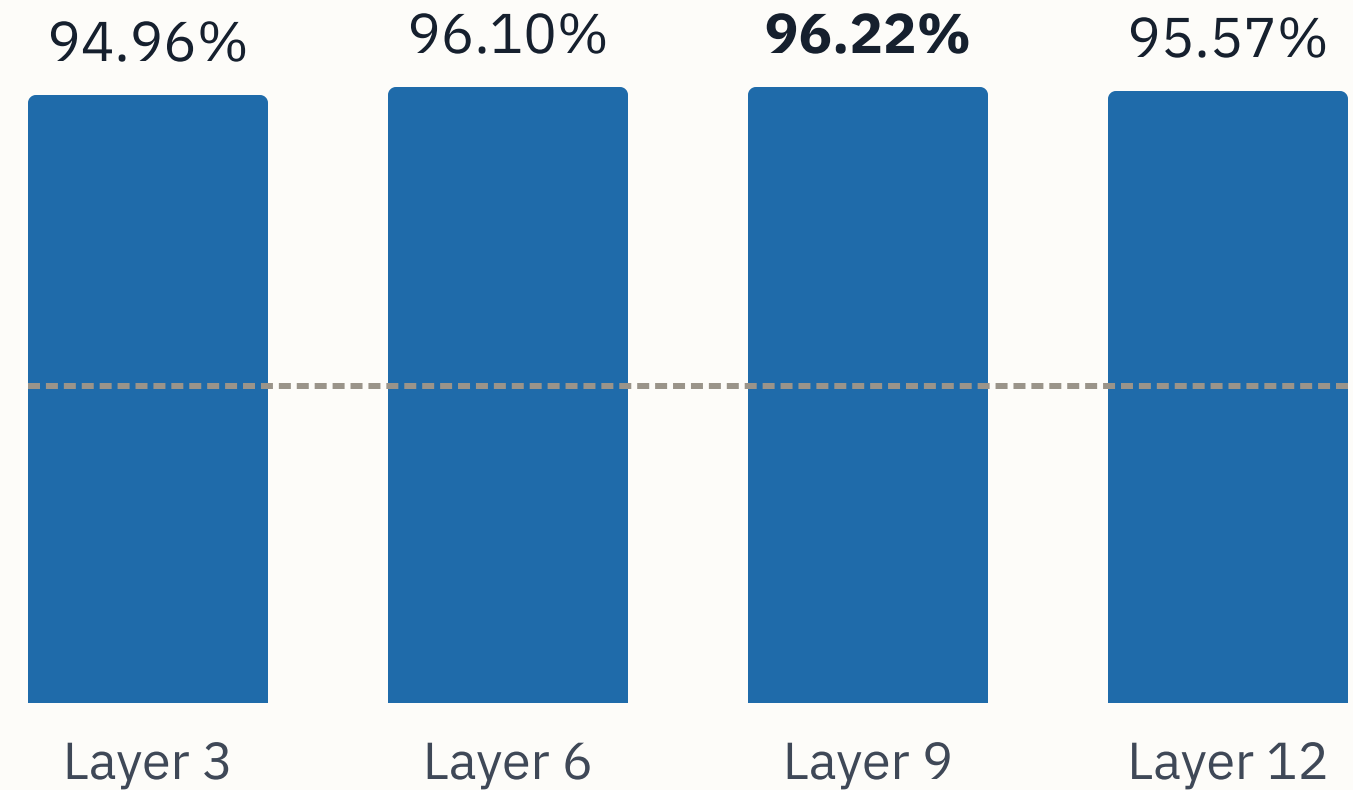
CIFAR-10: which object is it?

linear probe after each layer · dashed line = chance 10%



CIFAKE: real or AI-generated?

same probe, same frozen CLIP · dashed line = chance 50%



Back to fakes: real vs AI-generated images



CIFAKE: 60,000 real CIFAR-10 photos + 60,000 AI-generated look-alikes of the same 10 classes. 100,000 train / 20,000 test.

Top: real CIFAR-10 photos · Bottom: AI-generated (latent diffusion)

Method (same frozen CLIP ViT-B/16)	Accuracy	AUC	Trained weights
Zero-shot CLIP: “real photo” vs “AI-generated image”	49.18%	0.405	0
Linear probe on CLIP's final summary	95.56%	0.992	1,538
Linear probe on layer 9 (best single layer)	96.22%	0.994	1,538
Ours: frozen CLIP + side network, 1 epoch	96.92%	0.997	1,780,610

WRAP-UP

What this project shows

- 1 Frozen CLIP plus a side network with 2% of the weights reaches **96.42%** on CIFAR-10 in 5 epochs on a laptop GPU.
- 2 For object classes, CLIP's frozen features do most of the work: one linear layer gets 95.35%, and the side network adds about one point.
- 3 For real vs fake, CLIP needs training (zero-shot 49%), the signal sits in its middle layers, and our side network reaches **96.92%** on CIFAKE (AUC 0.997), above every single-layer probe.

WRAP-UP

Limits, and what to do next

Limits

- ViT-B/16, not the paper's ViT-L/14 (4 GB GPU)
- Face guidance and the frame branch were not tested: CIFAR-10 has no faces or video
- CIFAKE is real vs AI-generated objects, not face-swap videos
- One training run per setting, so no error bars yet

Next steps

- Train the full DFD-FCG on a face-video dataset and test on an unseen one: cross-dataset AUC, the paper's real claim
- Try ViT-L/14 on a larger GPU
- Use a validation split and 3 seeds per setting

Likely questions (1/3)

Why resize the 32×32 images to 224×224?

CLIP ViT-B/16 was trained on 224×224 images cut into 16×16 patches, a 14×14 grid. Its position embeddings expect that grid, so we upscale with bicubic interpolation, as CLIP's own preprocessing does.

Why freeze CLIP instead of fine-tuning it?

Fine-tuning all 86M weights needs far more memory and can overwrite CLIP's general knowledge. Freezing keeps that knowledge, and we train only 1.79M weights, 2% of the model.

Why a width of 192, and why layers 3, 6, 9 and 12?

192 is our own choice, 768 / 4: small matrices and 4 attention heads of 48. The four layers are evenly spaced, so the side network sees early, middle and late features. The paper reads every layer at full width.

Likely questions (2/3)

Is 96.42% believable after only 5 epochs?

Yes. I re-ran the saved model on all 10,000 test images: 9,642 correct. The test set was never used for training, and frozen CLIP with one linear layer already reaches 95.35%.

What does the side network actually add?

On CIFAR-10, about one point over a linear probe (95.35% → 96.42%). On real vs fake (CIFAKE) it beats every single-layer probe: 96.92% vs 96.22% for the best layer, AUC 0.997.

Why ViT-B/16 and not the paper's ViT-L/14?

ViT-L/14 has 303M weights and needs about 4–5 times the compute per image; my laptop GPU has 4 GB. ViT-B/16 is the same CLIP family, so the method is unchanged.

Likely questions (3/3)

What is an epoch, and why only 5?

One epoch is one pass over all 50,000 training images. Test accuracy was 94.1% after epoch 1 and rose only 0.3 points in the last epoch while train accuracy kept rising, so more epochs would mostly overfit.

CIFAR-10 has no fakes. Where is the deepfake part?

CIFAR-10 tests the method's core: adapting a frozen CLIP with a side network. For fakes, I ran the authors' trained model on face videos, and our model on CIFAKE, real vs AI-generated images.

What would you do next?

Run the full DFD-FCG on a face-video dataset and test on an unseen one (cross-dataset AUC), try ViT-L/14 on a larger GPU, and repeat runs with several seeds and a validation split.

Terms used in this talk

CLIP	OpenAI model trained on 400M image–text pairs to match pictures with captions
ViT-B/16	Vision Transformer, "Base" size, 16×16-pixel patches, 12 layers, 768 numbers per token
Token	one patch turned into a vector; plus one CLS token that summarises the image
Frozen	weights are fixed: no gradient, never updated during training
Side network	small trainable network that reads a frozen model's layers from the side
Epoch	one pass over all 50,000 training images
Linear probe	frozen features + one linear layer: a standard test of how good features are
Zero-shot	no training: CLIP picks the text prompt ("a photo of a cat") closest to the image
AUC	area under the ROC curve: chance that a random fake scores higher than a random real image
Overfitting	doing better on training images than on new ones

BACKUP

Where the code and results are

File (in C:\Users\hp)	What it is
dfd_fcg_cifar10_experiment.py	the model and the 5-epoch training run
dfd_side_network_cifar10.pth	trained weights, 96.42% (352 MB)
dfd_fcg_cifar10_768.py	side width 768 test (stopped early)
deepfake-clip-cifar10\eval_original.py	re-tests the saved weights
deepfake-clip-cifar10\baselines.py	zero-shot and linear-probe baselines
deepfake-clip-cifar10\train_sidenet.py	faster training, CIFAR-10 or CIFAKE
deepfake-clip-cifar10\results\	all numbers as JSON, plus logs
DFD-FCG\	authors' code, checkpoint, demo videos